

Start coding or [generate](#) with AI.

✓ Task

Create a baseline CNN model to predict heart disease based on the ECG image dataset located at ["/content/drive/MyDrive/Colab Notebooks/cropped dataset"](#). Do not include any preprocessing steps.

✓ Set up the environment

Subtask:

Import necessary libraries and set up the environment for working with image data and building a CNN.

Reasoning: Import the necessary libraries for building a CNN model with TensorFlow and Keras, including layers and the image data loader.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import image_dataset_from_directory
```

✓ Load the dataset

Subtask:

Load the ECG image dataset from the specified directory.

Reasoning: Load the image dataset from the specified directory using

`image_dataset_from_directory`.

```
dataset_dir = '/content/drive/MyDrive/Colab Notebooks/cropped dataset'
img_height = 600
img_width = 800
batch_size = 32

train_ds = image_dataset_from_directory(
    dataset_dir,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    seed=123,
    validation_split=0.3,
    subset='training'
)
```

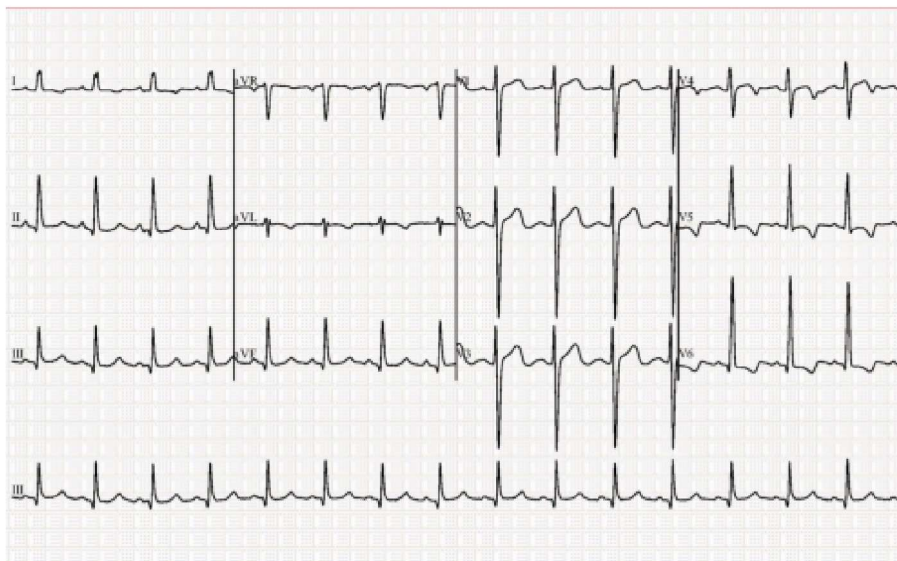
```
val_ds = image_dataset_from_directory(
    dataset_dir,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    seed=123,
    validation_split=0.3,
    subset='validation'
)
```

Found 2199 files belonging to 2 classes.
Using 1540 files for training.
Found 2199 files belonging to 2 classes.
Using 659 files for validation.

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

# Load the image
img = mpimg.imread('/content/drive/MyDrive/Colab Notebooks/cropped dataset/Abnormal/cropped_a

# Display the image
plt.imshow(img)
plt.axis('off') # Hide axes
plt.show()
```



✓ Create the cnn model

Subtask:

Define a baseline CNN model architecture using Keras.

Reasoning: Define the baseline CNN model architecture using Keras Sequential API by adding convolutional, pooling, flattening, and dense layers as specified in the instructions.

```

num_classes = len(train_ds.class_names)

model = keras.Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(img_height, img_width, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(num_classes, activation='softmax')
])

model.summary()

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: `super().__init__(activity_regularizer=activity_regularizer, **kwargs)`

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 598, 798, 32)	896
max_pooling2d (MaxPooling2D)	(None, 299, 399, 32)	0
conv2d_1 (Conv2D)	(None, 297, 397, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 148, 198, 64)	0
flatten (Flatten)	(None, 1875456)	0
dense (Dense)	(None, 128)	240,058,496
dense_1 (Dense)	(None, 2)	258

Total params: 240,078,146 (915.83 MB)

Trainable params: 240,078,146 (915.83 MB)

Non-trainable params: 0 (0.00 B)

✓ Compile the model

Subtask:

Configure the model for training by specifying an optimizer, loss function, and metrics.

Reasoning: Compile the Keras model with the specified optimizer, loss function, and metrics.

```

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

```

✓ Train the model

Subtask:

Train the CNN model using the loaded dataset.

Reasoning: Train the compiled CNN model using the training and validation datasets for a specified number of epochs and store the training history.

```
epochs = 10
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=epochs
)
```

```
Epoch 1/10
55/55 ————— 944s 14s/step - accuracy: 0.6019 - loss: 9919.0127 - val_accuracy:
Epoch 2/10
55/55 ————— 97s 2s/step - accuracy: 0.9852 - loss: 0.3162 - val_accuracy: 1.00
Epoch 3/10
55/55 ————— 99s 2s/step - accuracy: 1.0000 - loss: 9.4526e-04 - val_accuracy:
Epoch 4/10
55/55 ————— 96s 2s/step - accuracy: 1.0000 - loss: 4.0575e-04 - val_accuracy:
Epoch 5/10
55/55 ————— 146s 2s/step - accuracy: 1.0000 - loss: 2.4914e-04 - val_accuracy:
Epoch 6/10
55/55 ————— 140s 2s/step - accuracy: 1.0000 - loss: 2.7117e-04 - val_accuracy:
Epoch 7/10
55/55 ————— 143s 2s/step - accuracy: 1.0000 - loss: 1.4301e-04 - val_accuracy:
Epoch 8/10
55/55 ————— 139s 2s/step - accuracy: 1.0000 - loss: 1.1165e-04 - val_accuracy:
Epoch 9/10
55/55 ————— 95s 2s/step - accuracy: 1.0000 - loss: 9.2397e-05 - val_accuracy:
Epoch 10/10
55/55 ————— 145s 2s/step - accuracy: 1.0000 - loss: 1.1372e-04 - val_accuracy:
```

✓ Evaluate the model

Subtask:

Evaluate the performance of the trained model using the validation dataset.

Reasoning: Evaluate the trained CNN model using the validation dataset to assess its performance on unseen data.

```
loss, accuracy = model.evaluate(val_ds)
print(f"Validation loss: {loss}")
print(f"Validation accuracy: {accuracy}")
```

```
14/14 ————— 17s 1s/step - accuracy: 1.0000 - loss: 0.0013
Validation loss: 0.001014763955026865
Validation accuracy: 1.0
```

✓ Visualize training history

Subtask:

Visualize the training and validation accuracy and loss over epochs.

Reasoning: Plot the accuracy and loss for both training and validation datasets over the epochs to visualize the training progress and identify potential overfitting or underfitting.

```
import matplotlib.pyplot as plt

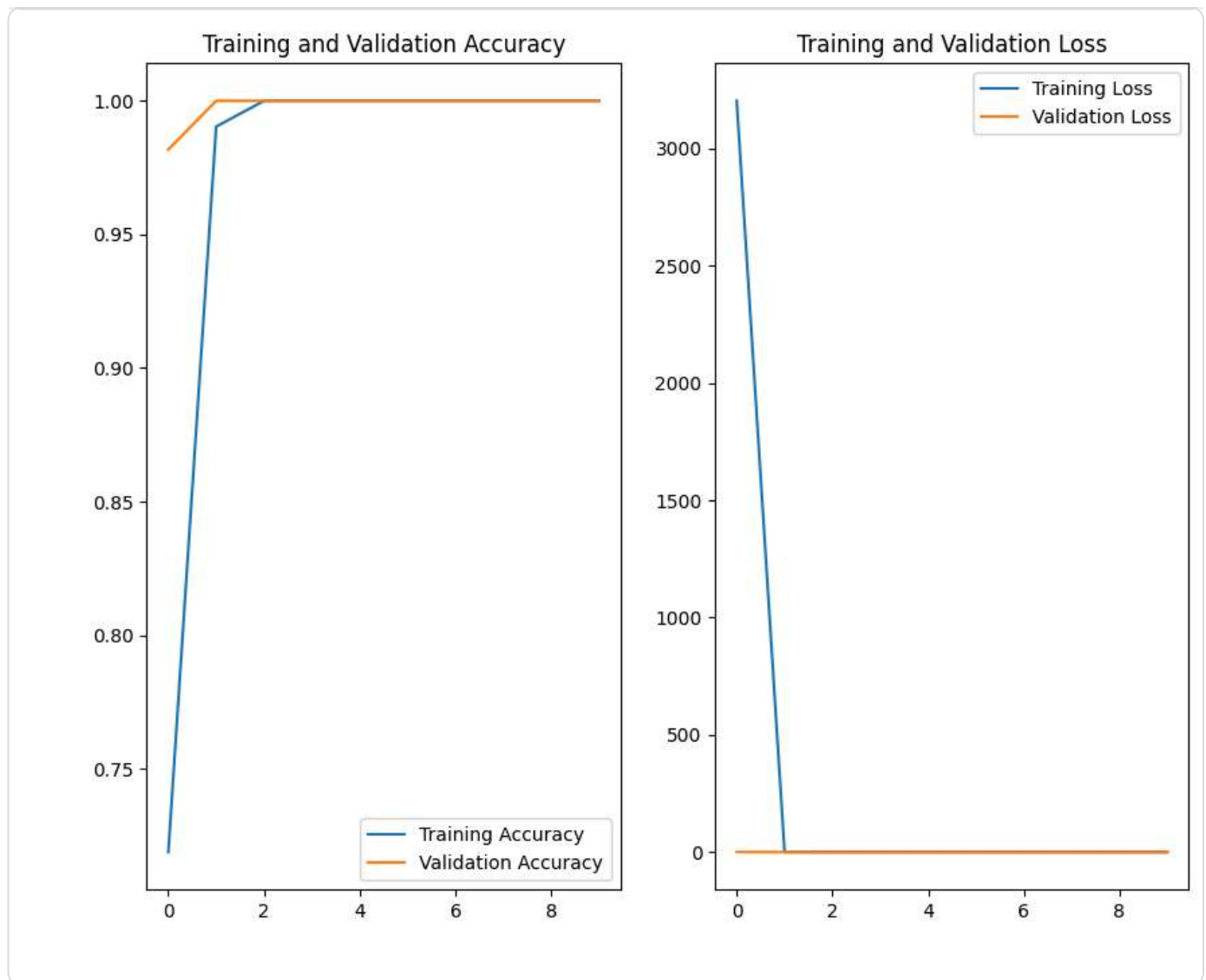
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

plt.figure(figsize=(10, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```



✓ Predict on a new image

Subtask:

Load a new ECG image and make a prediction using the trained model.

Reasoning: Load a new image, preprocess it to match the input requirements of the model, and then use the trained model to make a prediction.

```
import numpy as np
from tensorflow.keras.preprocessing import image

# TODO: Replace 'path/to/your/new_image.jpg' with the actual path to your new image
img_path = '/content/ecg.jpg'

img = image.load_img(img_path, target_size=(img_height, img_width))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0) # Create a batch
```

```
predictions = model.predict(img_array)

# Get the predicted class index
predicted_class_index = np.argmax(predictions[0])

# Get the class names from the training dataset
class_names = train_ds.class_names

# Get the predicted class name
predicted_class_name = class_names[predicted_class_index]

print(f"The predicted class is: {predicted_class_name}")
print(f"Prediction probabilities: {predictions[0]}")
```

```
1/1 ————— 0s 38ms/step
The predicted class is: Abnormal
Prediction probabilities: [1. 0.]
```